LL-Parser

BC George (HSBI)

Unless otherwise noted, this work is licensed under CC BY-SA 4.0.

Wiederholung

PDAs und kontextfreie Grammatiken

- Warum reichen uns DFAs nicht zum Matchen von Eingabezeichen?
- Wie könnnen wir sie minimal erweitern?
- Sind PDAs deterministisch?
- Wie sind kontextfreie Grammatiken definiert?
- Sind kontextfreie Grammatiken eindeutig?

Motivation

Was brauchen wir für die Syntaxanalyse von Programmen?

- einen Grammatiktypen, aus dem sich manuell oder automatisiert ein Programm zur deterministischen Syntaxanalyse erstellen lässt
- einen Algorithmus zum sog. Parsen von Programmen mit Hilfe einer solchen Grammatik

Themen für heute

- Syntaxanlyse
- Top-down-Analyse
- rekursiver Abstieg
- LL(k)-Analyse

Syntaxanalyse

Arten der Syntaxanalyse

Die Syntax bezieht sich auf die Struktur der zu analysierenden Eingabe, z. B. einem Computerprogramm in einer Hochsprache. Diese Struktur wird mit formalen Grammatiken beschrieben. Einsetzbar sind Grammatiken, die deterministisch kontextfreie Sprachen erzeugen.

- Top-Down-Analyse: Aufbau des Parse trees von oben nach unten
  - Parsen durch rekursiven Abstieg
  - tabellengesteuertes LL-Parsing
- Bottom-Up-Analyse: LR-Parsing

Bevor wir richtig anfangen...

Def.: Ein Nichtterminal A einer kontextfreien Grammatik G heißt *unerreichbar*, falls es kein $a, b \in (N \cup T)^*$ gibt mit $S \overset{*}{\Rightarrow} aAb$. Ein Nichtterminal A einer Grammatik G heißt *nutzlos*, wenn es kein Wort $w \in T^*$ gibt mit $A \overset{*}{\Rightarrow} w$.

Def.: Eine kontextfreie Grammatik $G = (N, T, P, S)$ heißt *reduziert*, wenn es keine nutzlosen oder unerreichbaren Nichtterminale in N gibt.

Bevor mit einer Grammatik weitergearbeitet wird, müssen erst alle nutzlosen und dann alle unerreichbaren Symbole eliminiert werden. Wir betrachten ab jetzt nur reduzierte Grammatiken.

Algorithmus: Rekursiver Abstieg

Hier ist ein einfacher Algorithmus, der (indeterministisch) top-down Ableitungen vom Nonterminal X aufbaut:

Eingabe: Ein Nichtterminal X und das nächste zu verarbeitende Eingabezeichen a.

```
RecursiveDescent(X, a) //X ∈ N, a ∈ T
    for a production, X → Y1Y2...Yn
        for i = 1 to n
            if Yi ∈ N
                RecursiveDescent(Yi, a)
            else if Yi ≠ a
                error
```

Abbildung 1: Recursive Descent-Algorithmus

Tabellengesteuerte Parser: LL(k)-Grammatiken

First-Mengen

$S \to A \mid B \mid C$

Welche Produktion nehmen?

Wir brauchen die "terminalen k-Anfänge" von Ableitungen von Nichtterminalen, um eindeutig die nächste zu benutzende Produktion festzulegen. k ist dabei die Anzahl der sog. *Vorschautoken*.

Def.: Wir definieren *First* - Mengen einer Grammatik wie folgt:

- $a \in T^*, |a| \leq k : First_k(a) = \{a\}$
- $a \in T^*, |a| > k : First_k(a) = \{v \in T^* \mid a = vw, |v| = k\}$
- $\alpha \in (N \cup T)^* \setminus T^* : First_k(\alpha) = \{v \in T^* \mid \alpha \overset{*}{\Rightarrow} w, \text{mit } w \in T^*, First_k(w) = \{v\}\}$

Linksableitungen

Def.: Bei einer kontextfreien Grammatik G ist die *Linksableitung* von $\alpha \in (N \cup T)^*$ die Ableitung, die man erhält, wenn in jedem Schritt das am weitesten links stehende Nichtterminal in α abgeleitet wird.

Man schreibt $\alpha \overset{*}{\Rightarrow}_l \beta$.

Follow-Mengen

Unless otherwise noted, this work is licensed under CC BY-SA 4.0.

Last modified: *87c95b9 (lecture: rework outcomes (02/LL-Theory), 2025-08-19)*